CS614 : NUMA Page Migration

Linux 6.1.4 Kernel Optimisation Report

PART 1	PART 2
O2 Optimisation : x86_64	**ROCM Optimisation : ARM64**
Deferred Batch IPI + Address-Ordered Sort	Read-Only Copy Migration Fast Path
VirtualBox i5-1340P (baseline+optimised)	ARM64 / Linux 6.1.4 / numa=fake=2
Akshat Xeon 6226R + Cezan i7-13620H	Stall elimination for read-only pages

Group Name : team_name.ko

Group Members:
Aayush Kumar (230027), Cezan Damania (230310), Yogit (211207)

x86_64 Kernel Optimisation

O2: Deferred Batch IPI + Address-Ordered Sort

1. The Problem : IPI Shootdown Bottleneck

1.1 Background: The Five-Stage Migration Pipeline

Every NUMA page migration in Linux 6.1.4 passes through five sequential stages. The kernel records nanosecond timestamps for each stage transition, exposing the per-page cost breakdown:

Stage	x86_64 mean	% of total	What happens	Cost driver
Lock	48 ns	0.7%	Acquire folio_lock(src) + folio_lock(dst)	Negligible
Unmap	**2,609 ns**	**41.0%**	ptep_clear_flush() — clear PTE + INVLPG + flush_tlb_others() IPI to all remote CPUs	IPI shootdown
Copy	1,380 ns	21.7%	copy_page(src, dst) — memcpy of 4KB	Memory BW
Remap	340 ns	5.3%	remove_migration_ptes() — install new PTE	PTE write
Unlock	180 ns	2.8%	folio_unlock(src) — wake waiting threads	Negligible

Unmap is 41% of total wall-clock time in quiescent single-thread runs. Under concurrent load it rises to 97–99% of total time.

1.2 Sub-Component Decomposition (T1–T5 Analysis)

To isolate the cause of Unmap's dominance, the kernel was instrumented with per-CPU nanosecond timing brackets inside try_to_migrate_one(). Five independent tests (T1–T5) decomposed the Unmap stage into four non-overlapping components:

ID	Hypothesis	Test	Verdict	Evidence
H1	IPI shootdown is dominant	T5 rmap sub-timing	**CONFIRMED**	98.5% of try_migrate under load

H2	PTE spinlock contention	T4 load delta	**RULED OUT**	No ptl_ns delta under load
H3	kswapd rwsem contention	T3 outlier clustering	**RULED OUT**	Outliers scattered (91% span), not clustered
H4	Page-table cache thrash	T2 quintile analysis	**RULED OUT**	Q5/Q1 ≤ 1.0 — warm-up, not thrash

1.3 The Quantified Root Cause

T5 on the VirtualBox i5-1340P machine (baseline kernel, no optimisation) revealed the exact cost breakdown per page in two scenarios:

Component	Quiescent	Under load	Multiplier	Physical cause
H1 ipi_wait_ns (flush_tlb_others)	632 ns (25%)	**139,538 ns (98.5%)**	**221×**	Busy CPUs take 53 µs to ACK IPI
H2 PTE body (page_remove_rmap)	894 ns (36%)	1,291 ns (0.9%)	1.4×	LOCK CMPXCHG + set_pte_at
H3 rwsem (down_read)	277 ns (11%)	0 ns (0%)	—	Cache-hot from workers
Structural walk (rmap + page table)	683 ns (28%)	885 ns (0.6%)	1.3×	4-level PGD walk per page
try_migrate total	**2,486 ns**	**141,714 ns**	**57×**	IPI dominates under concurrent load

The IPI cost explodes under load because the 3 worker CPUs are executing tight memory-access loops. When the migrator calls flush_tlb_others(), each worker CPU must complete its in-flight L3 cache miss (~100 ns DRAM latency on real hardware, longer in a VM) before taking the interrupt. With 3 workers each adding ~53 µs, the total IPI cost per page becomes 159 µs, 361× the quiescent cost.

The structural walk (H4, 683 ns quiescent) was identified as a secondary target: 2,048 pages migrating in arbitrary order cause repeated cold 4-level page-table walks. Sorting pages by virtual address order would keep PGD/PUD/PMD table entries hot in L1 cache, reducing subsequent walks from 4 levels to 1.

2. The Optimisation : O2 Design and Implementation

O2 combines two coordinated changes: O2a (deferred batch IPI) addresses the 98.5% problem, and O2b (address-ordered sort) addresses the 28% structural walk cost.

2.1 O2a : Deferred Batch TLB Shootdown

2.1.1 How the Standard Path Works

The standard migration path calls try_to_migrate() once per page. Inside try_to_migrate_one(), each PTE is cleared with ptep_clear_flush() which does three operations:

- LOCK CMPXCHG : atomically clear the PTE. No IPI. ~5 ns.
- INVLPG : flush the local CPU's TLB entry for this VA. No IPI. ~5 cycles.
- flush_tlb_others() : send an IPI to every remote CPU that has this mm loaded in CR3. The migrator blocks until ALL remote CPUs ACK. This is the bottleneck: 159 µs per page under load

For a 512-page batch, this means 512 × 159 µs = 81.5 ms of IPI wait. The migrator cannot proceed with the next page until all remote CPUs have acknowledged. Workers are interrupted 512 separate times.

2.1.2 The Deferred Batch Approach

O2a phase-separates migration into three stages so N pages share ONE IPI round-trip:

Phase	What runs	How it works	IPI sent?
1	Batch unmap (N pages)	For each page: ptep_get_and_clear() clears PTE atomically (LOCK CMPXCHG). No INVLPG, no IPI. set_tlb_ubc_flush_pending() registers the mm for a deferred flush. Migration swap entry installed — workers fault immediately and sleep.	**NO — deferred**
2	Single batch flush	try_to_unmap_flush() → arch_tlbbatch_flush() issues ONE set of IPIs covering ALL pages unmapped in Phase 1. On x86: flush_tlb_mm_range() to all CPUs that loaded this mm. One IPI roundtrip regardless of batch size N.	**YES — once only**

3	Copy + remap (N pages)	move_to_new_folio() copies src→dst. remove_migration_ptes() installs new PTEs. Locks released. Src pages freed. Workers wake when folio_unlock(src) is called.	No

2.1.3 Correctness Argument

The safety of deferring the TLB flush rests on the migration swap entry. After ptep_get_and_clear() clears the PTE (Present=0) and the migration swap entry is installed, any worker accessing that VA immediately takes #PF → do_swap_page() → migration_entry_wait() → sleeps until folio_unlock(src). This sleep happens regardless of whether the remote TLB flush has occurred. Therefore:

- Workers cannot access stale physical pages through a stale TLB entry — they are blocked by the migration entry, not by TLB coherence.
- Src physical page content is valid (unmodified) until Phase 3 copy starts. A CPU reading through a stale TLB entry before Phase 2 flush reads correct data.
- Src pages are never freed until after Phase 2 flush. No stale TLB entry can reach reallocated memory.
- Phase 2 flush must complete before Phase 3 copy. This is enforced by the phase structure — try_to_unmap_flush() is called and returns before move_to_new_folio() is called for any page in the batch.

One critical implementation detail: all N src and dst folios remain locked throughout Phases 1 and 2. If src were unlocked and freed between Phase 1 and Phase 2, a remote CPU's stale TLB entry could access the freed physical frame after it had been reallocated. The lock holds prevent this.

2.1.4 Kernel Changes

Two files modified:

File	Change
`mm/rmap.c`	(1) TTU_BATCH_FLUSH added to the allowed flags set in try_to_migrate(). The original WARN_ON_ONCE guard explicitly rejected this flag. (2) In try_to_migrate_one(), the ptep_clear_flush() call is wrapped in an if/else: when TTU_BATCH_FLUSH is active, uses ptep_get_and_clear() + set_tlb_ubc_flush_pending() instead. Falls through to original path when TTU_BATCH_FLUSH is absent : no behaviour change for existing callers.

`mm/migrate.c`	(1) struct mig_batch_entry : per-page state tracking src/dst folios, anon_vma reference, and per-phase timing. Heap-allocated (kmalloc_array, MIG_BATCH_SIZE=32) to avoid >1024 byte stack frame. (2) migrate_pages_deferred_ipi() , 300-line batch function implementing Phases 1/2/3. Called from migrate_pages() before the standard retry loop, serving all callers: move_pages() syscall, NUMA balancing (MR_NUMA_MISPLACED), and compaction. (3) mig_timing_store() called in Phase 3 with ipi_wait_ns set to amortised flush cost (total_flush_ns / n_pages) so the debugfs CSV correctly captures batch-path IPI cost.

2.2 O2b : Address-Ordered Migration

The structural walk cost (683 ns, 28% of quiescent try_migrate) comes from the 4-level page-table walk inside page_vma_mapped_walk(): PGD → PUD → PMD → PTE. When pages migrate in arbitrary order, each walk starts cold — all three upper table levels must be fetched from main memory.

For sequentially-allocated anonymous pages, consecutive PFNs (page frame numbers) correspond to consecutive virtual addresses. Pages within the same 2MB VA region share PGD, PUD, and PMD table entries. If migrations are sorted by PFN, the walk for page N+1 in the same PMD region only needs to fetch the PTE — the upper three levels are still hot in L1 cache from page N's walk. Effective walk cost: ~137 ns instead of ~683 ns for these pages.

Implementation: a single list_sort() call with an 8-line PFN comparator in migrate_pages(), applied before the migration loop. Cost: ~10 µs for 512 pages (O(N log N)). Benefit: 37–75% reduction in structural walk cost depending on how many consecutive pages fall within the same PMD region.

O2b change	Code
PFN comparator	`static int mig_pfn_cmp(void *priv, const struct list_head *a, const struct list_head *b) { return page_to_pfn(entry_a) < page_to_pfn(entry_b) ? -1 : 1; }`
Sort call in migrate_pages()	`list_sort(NULL, from, mig_pfn_cmp); // before retry loop`

3. Measured Results : x86_64

Evaluation on three machines with identical O2 kernel patches, contrasting VirtualBox (real IPI cost) against KVM (IPI absorbed by hypervisor):

Machine	VirtualBox / i5-1340P	Akshat / Xeon 6226R / KVM	Cezan / i7-13620H / KVM
IPI mechanism	Software APIC — real IPI cost	KVM intercepts — absorbed	KVM intercepts — absorbed
IPI visible to O2?	YES — O2 eliminates it	NO — already zero	NO — already zero
Bench A throughput	422 MB/s (baseline)	948 MB/s	**1,611 MB/s**
T5: IPI % quiescent	25.4% (baseline) → 22.5% (O2)	0% — KVM absorbs	Not run (older script)
T2: Q5/Q1	2.32→0.69 (O2b fixed thrash)	0.992 (flat — O2b working)	0.931 (flat)

3.1 Benchmark A: Quiescent Baseline vs O2 (VirtualBox, Real IPI)

The VirtualBox machine is the only environment where real IPI cost is visible, providing the direct before/after comparison for O2a. All three machines show O2b's structural walk improvement.

Metric	Baseline	After O2	Improvement	Driven by
try_migrate_ns median	2,486 ns	**1,488 ns**	**1.67×**	O2a + O2b
ipi_wait_ns median	**632 ns (25.4%)**	**335 ns (22.5%)**	**1.89×**	O2a batch flush
PTE body	894 ns (36.0%)	**480 ns (32.2%)**	**1.86×**	O2b locality
rwsem	277 ns (11.1%)	244 ns (16.4%)	1.14×	Incidental
Structural walk (O2b target)	**683 ns (27.5%)**	**429 ns (28.9%)**	**1.59×**	O2b PMD locality
unmap_ns total	**2,609 ns**	**1,642 ns**	**1.59×**	Both optimisations
total_ns (end-to-end)	5,781 ns	**4,633 ns**	**1.25×**	Unmap reduction

Structural walk: 683 ns → 429 ns (1.59×) — O2b confirmed on VirtualBox (real IPI visible)

3.2 Benchmark A: Structural Walk Across All Three Machines

O2b is architecture-neutral and visible regardless of IPI mechanism. Akshat's Xeon provides the cleanest O2b measurement (CV=0.34, zero T3 outliers):

Machine	try_migrate median	Structural walk (outside-ptl)	vs 683 ns baseline
VirtualBox i5 (baseline, no O2)	2,486 ns	**683 ns (27.5%)**	Reference
VirtualBox i5 (with O2)	**1,488 ns**	**429 ns (28.9%)**	**1.59× better**
Akshat Xeon 6226R (O2, KVM)	**809 ns**	**270 ns (33.4%)**	**2.02× better ✓**

The Xeon achieves a greater walk reduction (2.02×) because the 2.9 GHz clock speed and larger L1/L2 cache mean PMD entries stay warmer across the 32-page batch. The VirtualBox VM has additional overhead from hypervisor page-table traversal on the host side that adds noise to the walk measurement.

3.3 Benchmark E: Concurrent Load : Primary Config (rand_rmw_t4_c512)

The primary Bench E configuration: random RMW, 2 workers on NUMA node 1, chunk=512 pages, 30-second migration window.

Key finding: With proper NUMA vCPU pinning, O2 makes migration NET-BENEFICIAL : workers run faster during migration than baseline, because locality gain outpaces disruption.

Metric	VirtualBox i5 (O2)	Akshat Xeon (O2)	Cezan i7 (O2)
Disruption factor	**0.767**	**1.093 ✓**	**1.098 ✓**
Phase 3 recovery	**1.003 (full recovery)**	**1.194 (19% faster)**	**1.159 (16% faster)**
p999 during migration	41,858 ns	**586 ns**	838 ns
p999 spike ratio	42×	**1.2×**	**1.1×**
Migration rate	2,848 pages/s	15,848 pages/s	**33,800 pages/s**
IPI mechanism	Software APIC (real IPI)	KVM intercept (absorbed)	KVM intercept (absorbed)

Why VirtualBox shows lower disruption (0.767) than KVM machines (1.093/1.098): On VirtualBox, real IPI shootdowns still interrupt workers during each batch flush (one IPI per 32 pages instead of per page ,a 32× reduction in interruptions). Workers still stall for the batch flush but far less often. On KVM, the hypervisor absorbs IPIs entirely, so the only disruption workers see is the migration-PTE fault latency, which is overwhelmed by the locality gain from pages moving to the local NUMA node. Both results demonstrate O2 working correctly.

3.4 Benchmark E: All 7 Configurations : Akshat and Cezan

x86 vs x86 comparison, same kernel, same NUMA topology structure, different CPU performance:

Config	Akshat disrupt	Akshat recov.	Cezan disrupt	Cezan recov.	Agreement
rand_rmw_t4_c512	**1.093**	1.194	**1.098**	1.159	✓✓
seq_read_t4_c512	0.878	0.913	1.052	1.098	Cezan>1, Akshat<1
rand_read_t4_c512	0.865	0.814	1.033	1.010	Cezan>1, Akshat<1
rand_rmw_t1_c512	**1.513**	**1.598**	1.001	1.009	Both >1
rand_rmw_t8_c512	1.094	1.130	1.011	1.032	**Both >1**
rand_rmw_t4_c1	0.999	**1.499**	0.963	0.993	Near-neutral

The c64 anomaly (Cezan 0.527 vs Akshat 0.897) is explained by migration rate: Cezan's i7 migrates at 17,976 pages/s with chunk=64, completing all 131,072 pages in ~7.3 seconds. The remaining 22.7 seconds of Phase 2 have workers accessing a split buffer (pages distributed between both nodes). Akshat's Xeon migrates at 2,060 pages/s (chunk=64), completing only 61,824 pages in 30 seconds, a partial migration that explains its disruption of 0.897 (workers hit remote pages throughout). This is a migration-rate effect, not a kernel correctness issue.

4. Why Results Differ Across Machines

4.1 VirtualBox vs KVM: The IPI Absorption Difference

The disruption factor difference (VirtualBox 0.767 vs KVM 1.09+) is not a discrepancy — it is the expected consequence of how each hypervisor handles TLB shootdown IPIs:

Aspect	VirtualBox	KVM with -cpu host
IPI handling	Virtualised APIC: guest IPI propagates to real hardware. Remote vCPU takes real interrupt.	Hypervisor intercepts flush_tlb_others(). Handles TLB coherence in host kernel without descheduling guest vCPUs.
Worker disruption	Workers interrupted by IPI during each batch flush (1 per 32 pages, vs 1 per page without O2)	Workers never interrupted — hypervisor makes IPI cost zero
O2a effect	Visible: 32× fewer IPI interruptions → 0.767 disruption (up from 0.10–0.25 without O2)	Invisible for disruption — already zero. Locality gain from page migration dominates: disruption > 1.0
O2b effect	Visible: walk 683→429 ns (1.59×)	Visible: walk 683→270 ns (2.02×) — Xeon's larger cache amplifies the PMD locality benefit

Both environments confirm O2 working correctly. VirtualBox provides direct evidence of IPI cost reduction. KVM provides evidence of the downstream application benefit — when IPI overhead is eliminated, migration becomes transparent or net-positive because the locality gain (workers get local memory) exceeds any residual disruption.

PART 2 ARM64 Kernel Optimisation — ROCM: Read-Only Copy Migration Fast Path

5. The Problem : Migration-Entry Stall for Read-Only Pages

5.1 The Standard Migration Pipeline on ARM64

ARM64's hardware TLBI (Translation Lookaside Buffer Invalidation) broadcast eliminates the software IPI shootdown problem that dominates x86. TLBI VAE1IS + DSB ISH atomically invalidates TLB entries across all cores without software interrupts. As a result, the ARM64 bottleneck is different:

Stage	ARM64 mean	% of total	Notes
Lock	28 ns	2.1%	Negligible
Unmap	420 ns	31.4%	TLBI VAE1IS + DSB ISH per PTE — hardware broadcast, no software IPI
Copy	**610 ns**	**46.2%**	Dominant — copy_page() under a migration entry. Faulting threads sleep here.
Remap	120 ns	9.0%	remove_migration_ptes() — ROCM eliminates this entirely
Unlock	40 ns	3.0%	Wake faulting threads via folio_unlock(src)

5.2 Why Faulting Threads Stall Unnecessarily

When a worker accesses a page under migration, it hits the migration swap entry (Present=0) and calls migration_entry_wait() — sleeping until folio_unlock(src). The thread's stall window covers the full Unmap + Copy + Remap duration: 875 ns per page in Benchmark A.

For read-only file-backed pages, this stall is unnecessary. The page content is stable — no writes can occur to a read-only mapping. A worker could safely read the source page content at any point during the migration. The migration entry exists to maintain TLB consistency, but for read-only pages a different approach is possible: copy first, then atomically swap the PTE from old to new translation without ever installing a migration swap entry.

\

	Standard pipeline (all pages)	ROCM pipeline (read-only pages)
Order	Lock → Unmap → Copy → Remap → Unlock	Lock → Copy → PTE-swap → Unlock
Thread stall window	**Unmap+Copy+Remap = 875 ns. Thread sleeps in migration_entry_wait() through entire copy.**	**PTE-swap only = 462 ns. No migration entry. Thread finds new PTE immediately.**
Migration entry installed?	YES — Present=0 migration swap entry. All accesses fault and sleep.	NO — src PTE live until atomic swap. Threads read src content during copy.
Remap stage	75 ns — remove_migration_ptes() needed	**0 ns — eliminated entirely**

6. The Optimisation : ROCM Design and Implementation

6.1 Gate Condition

ROCM applies only when folio_rmap_all_readonly() returns true, every PTE mapping the page is read-only (PROT_READ) and the folio is file-backed (not anonymous). This covers the common case of shared libraries, read-only memory-mapped files, and clean page-cache pages. Anonymous pages (malloc/mmap without MAP_PRIVATE) always go through the standard pipeline.

6.2 The ROCM Protocol

Phase 1 — Copy with src PTEs live:

- Allocate dst page and lock it.
- copy_page(src, dst) — copy content while src PTE is still present and readable. Workers can continue reading src during this copy (no stall).
- No migration entry installed. No faulting threads blocked.

Phase 2 — Atomic PTE swap:

- For each PTE mapping src: atomically replace old translation (src phys) with new translation (dst phys) in a single CMPXCHG operation.
- Issue TLBI VAE1IS + DSB ISH — one hardware TLB invalidation per PTE swap. This is the stall window: 462 ns (one TLBI round-trip on this hardware).
- Workers that access during the TLBI window get the new translation immediately — no sleep, no migration_entry_wait().

Phase 3 — Cleanup:

- folio_unlock(src). No threads are waiting (none were blocked).
- put_page(src) — free src frame.

6.3 Why the Stall is Not Zero

The original projection was ~70 ns (based on the Remap stage cost — installing a PTE into a non-present slot, which requires no TLBI). ROCM's actual stall is 462 ns because the PTE swap replaces a live translation, which requires a full TLBI VAE1IS + DSB ISH broadcast. This cannot be avoided: hardware requires that all cores acknowledge the translation change before it is visible. The theoretical lower bound is one TLBI round-trip (~420 ns on this hardware). The measured 462 ns is near-optimal.

6.4 Kernel Changes

File	Change
`mm/migrate.c (ARM64)`	ROCM gate: folio_rmap_all_readonly() check before standard unmap call. New function ro_copy_and_swap(): copies src→dst then calls try_to_migrate_ro() to swap PTEs without installing a migration swap entry. Added rocm_path column to mig_timing_record for identification.
`mm/rmap.c (ARM64)`	New try_to_migrate_ro(): rmap walk to find each PTE, CMPXCHG to swap old→new translation, TLBI VAE1IS + DSB ISH per PTE. Returns failure if any PTE is found to be writable (falls back to standard path).

7. Measured Results, ARM64 ROCM

All results from controlled comparison: identical workload (2 workers, random reads, 32 MB file-backed buffer), only migration pipeline differs.

7.1 Stage Timing Comparison

Stage	Standard mean (ns)	ROCM mean (ns)	Change
Lock	21	23	+10% (noise)
Unmap / PTE-swap	383	502	ROCM: PTE-swap replaces unmap
Copy	**416 ns — stall window**	**406 ns — NOT a stall**	Moved before stall window

Remap	75 ns	**0 ns**	**ELIMINATED ✓**
Unlock	22	25	~same
Total wall-clock	1,097 ns	1,330 ns	+21% (acceptable)

ROCM's total wall-clock is 21% higher than standard because the copy occurs before the stall window (before any PTE is cleared), meaning it competes with live worker accesses to src. The standard pipeline copies under a migration entry when workers are blocked, an artificial bandwidth boost. The 21% total overhead is the cost of zero-stall migration.

7.2 Application Stall Comparison (Primary Metric)

Metric	Standard pipeline	ROCM pipeline
Application stall window	**875 ns (unmap+copy+remap)**	**462 ns (PTE-swap only)**
Stall reduction	—	**1.89× faster stall ✓**
Remap stage cost	75 ns	**0 ns — eliminated**
rocm_path=1 records	0 / 8192 (0%)	**8192 / 8192 (100%)**
Migration rate	46,704 pages/s	**84,154 pages/s**
Migration rate improvement	—	**1.80× faster**
Mean stall duration (Bench E)	54,869 ns	**26,729 ns**
Mean stall improvement	—	**2.05× shorter mean stall**
Disruption factor	1.020	1.077
Stalls per migrated page	1.74	**1.60 (1.09× fewer)**

Application stall: 875 ns → 462 ns (1.89×). Remap eliminated. Migration 1.8× faster. Mean stall duration 2.05× shorter.

7.3 Why ROCM Stall is 462 ns and Not Zero

Component	Analysis

Original projection	~70 ns (based on Remap stage cost — installing PTE into non-present slot with no TLBI needed)
Actual stall	462 ns (TLBI VAE1IS + DSB ISH roundtrip)
Why different	ROCM swaps a live translation (old → new), not an absent-to-present transition. Live translation swap requires TLBI to ensure all cores see the new translation before any access. The projection assumed a non-present → present PTE install (no TLBI). ROCM requires present → present TLBI. Cannot be avoided.
Theoretical minimum	~420 ns = one TLBI VAE1IS round-trip on this hardware. Measured 462 ns is 10% above minimum — near-optimal.

Appendix A : x86_64 O2 Artifact

Artifact for O2a (Deferred Batch TLB Shootdown) and O2b (Address-Ordered Migration). Evaluated on x86_64 Linux 6.1.4 with emulated NUMA (numa=fake=2).

A.1 File Inventory

All files must be placed in one flat directory alongside run_x86_full.sh.

File	Size	Description
`run_x86_full.sh`	11 KB	Main automation script. 9 steps: preflight → build → Bench A-D → timing plots → T5/T2/T3 analysis → Bench D plots → Bench E → O2a verification → summary. Run as: sudo bash run_x86_full.sh [--quick] [--skip-benche]
`run_t1.sh`	9 KB	T1 CPU hotplug IPI isolation driver. Offlines CPUs one-by-one and re-runs Bench A at each count. Invoked automatically by run_x86_full.sh Step 5 if present.

mig_bench_x86.c	42 KB	Userspace Benchmarks A, B, C, D. Bench A: 4KB quiescent timing (512 and 2048 pages). Bench B: 2MB THP migration. Bench C: shared-page rmap scaling (degrees 1–64). Bench D: application-visible migration stall.
mig_bench_e_x86.c	39 KB	Benchmark E: 7 concurrent-load configurations sweeping access pattern (rand/seq), operation (RMW/read), thread count (t1/t4/t8), and chunk size (c1/c64/c512). Measures worker disruption and p999 latency during live migration.
analyze_unmap.py	33 KB	T1–T5 sub-component decomposition. Flags: --t2 (cache thrash), --t3 (rwsem outliers), --t4 (load delta), --t5 (full breakdown), --all. Reads timing CSVs and prints IPI/PTL/rwsem/walk ns breakdown.
analyze_timing.py	31 KB	Stage breakdown analysis and plotting. Reads timing_*.csv files; generates pie chart and boxplot PNG files into results_x86/plots/. Called automatically in Step 4.
analyze_bench_e_x86.py	31 KB	Bench E analysis and plotting. Reads bench_e_workers_*.csv; generates disruption factor and p999 spike charts into results_x86/plots/. Called automatically after Step 7.
migrate.c	96 KB	Optimised kernel source for mm/migrate.c. Contains O2a (migrate_pages_deferred_ipi, MIG_BATCH_SIZE=32, mig_batch_entry) and O2b (mig_pfn_cmp + list_sort call). Must be copied to linux-6.1.4/mm/migrate.c before kernel build.
rmap.c	78 KB	Optimised kernel source for mm/rmap.c. Contains TTU_BATCH_FLUSH flag, deferred ptep_get_and_clear branch, and per-CPU sub-timing brackets (ipi_wait_ns, ptl_wait_ns, rwsem_wait_ns producing the 25-column debugfs CSV). Must be copied to linux-6.1.4/mm/rmap.c.

migrate_x86.patch	44 KB	Unified diff of mm/migrate.c against stock Linux 6.1.4. Alternative to copying the full file: cd linux-6.1.4 && patch -p1 < migrate_x86.patch
rmap_x86.patch	7 KB	Unified diff of mm/rmap.c against stock Linux 6.1.4. Alternative to the full file.
README	13 KB	Artifact README. Road map for evaluation, required files table, setup instructions, troubleshooting table, expected outcomes.

A.2 Setup Instructions

Kernel Compilation

```
# Copy optimised sources:
cp migrate.c  ~/linux-6.1.4/mm/migrate.c
cp rmap.c       ~/linux-6.1.4/mm/rmap.c
# Or apply patches:
cd ~/linux-6.1.4
patch -p1 < /path/to/migrate_x86.patch
patch -p1 < /path/to/rmap_x86.patch

# Configure and build (Human: ~5 min | Compute: ~25 min):
cp /boot/config-$(uname -r) .config && make olddefconfig
scripts/config --disable SYSTEM_TRUSTED_KEYS
scripts/config --disable SYSTEM_REVOCATION_KEYS
make -j$(nproc)
sudo make modules_install && sudo make install && sudo reboot

# Verify after reboot:
uname -r                                                  # 6.1.4
sudo ls /sys/kernel/debug/mig_timing                     # must exist
head -1 /sys/kernel/debug/mig_timing | tr ',' '\n' | wc -l  # must print 25
grep -c migrate_pages_deferred_ipi /proc/kallsyms    # must be ≥ 1
```

Build Userspace Benchmarks

```
# Human time: < 1 minute
gcc -O2 -Wall -o mig_bench   mig_bench_x86.c   -lnuma -lpthread
gcc -O2 -Wall -o mig_bench_e mig_bench_e_x86.c -lnuma -lpthread -lm
```

A.3 Features and Evaluation Table

For each feature: test scenario, parameters, pointer to automation script, objective, and expected outcome.

Experiment	Threads	Parameters	Script	Objective	Expected outcome

Bench A 4KB quiescent	1 migrator	512 pages 2048 pages node 0→1 no concurrent load	run_x86_full.sh Step 3 sudo ./mig_bench	Measure per-page migration stage costs. Verify O2b reduces structural walk from 683 ns baseline. Produce quiescent CSV for T2/T3/T5.	timing_4kb_2048.csv: 2048 records, 25 columns. try_migrate median 800–1500 ns. Walk (outside-ptl) 250–550 ns.
Bench B 2MB THP	1 migrator	32 THPs 128 THPs node 0→1	Step 3 (auto) sudo ./mig_bench	THP migration cost profile. Confirm copy-dominated at 2MB scale.	timing_2mb_*.csv. Copy BW 300–2800 MB/s. Copy > 90% of total time.
Bench C Shared pages	1 migrator	64 pages deg 1,2,4,8,16,32,64 node 0→1	Step 3 (auto) sudo ./mig_bench	Measure rmap walk scaling with sharing degree. Confirm O(mapcount) unmap cost growth.	7 CSVs timing_shared_deg001..064.csv. Unmap rises sub-linearly with degree.
Bench D Downtime	1 accessor 1 migrator	400,000 RDTSC samples same-node + cross-node threshold: 5,000 ns	Step 3 (auto) sudo ./mig_bench	Measure app-visible stall when thread accesses a page under migration. Confirm migration-entry is bottleneck.	downtime_samenode.csv: 6–18 stall events, max stall 42–310 µs. Normal access mean 13–36 ns.
T2 Cache thrash	—	timing_4kb_2048.csv 5 quintiles of seq num	Step 5 auto analyze_unmap.py --t2	Rule out H4: page-table cache thrash. Expect flat or decreasing cost over sequential page number.	Q5/Q1 < 1.10. Output: H4 NOT SUPPORTED.

T3 rwsem outliers	—	timing_4kb_2048.csv 5× median threshold	Step 5 auto analyze_unmap.py --t3	Rule out H3: kswapd rwsem contention causing burst outlier spikes.	Outliers < 2%, span > 40% of range (scattered). Output: H3 AMBIGUOUS or RULED OUT.
T5 Sub-component	—	timing_4kb_2048.csv 25 CSV columns required	Step 5 auto analyze_unmap.py --t5	Isolate H1 (IPI), H2 (PTL body), H3 (rwsem), and structural walk (outside-ptl) in nanoseconds per page.	IPI 0–25% quiescent. Walk 27–35%. PTE body 32–63%. CV < 0.5 on clean run.
Bench E config 0 rand_rmw_t4_c512	2 workers (clamped) 1 migrator	512 MB buffer chunk=512 pages 10s/30s/10s phases	Step 7 run_x86_full.sh --quick or: sudo ./mig_bench_e 0	Primary disruption metric. Validate O2a reduces worker disruption vs unoptimised baseline of 0.10–0.25.	Disruption factor ≥ 0.60. Phase 3 recovery ≥ 0.90. p999 < 50,000 ns. O1 CONFIRMED in log.
Bench E config 6 rand_rmw_t4_c1	2 workers 1 migrator	512 MB buffer chunk=1 page 10s/30s/10s	Step 7 sudo ./mig_bench_e 5	Expose per-page overhead with batch size=1 (no O2a batching). Shows p999 spike characteristic of unoptimised per-page IPI.	Disruption ≈ 0.96–1.00. p999 spike 10–23×. Migration rate ~34 pages/s.
Bench E all 7	1–2 workers 1 migrator	7 configs sweep rand/seq × rmw/read t1/t4/t8 × c1/c64/c512	Step 7 full run sudo ./mig_bench_e	Full sweep across access patterns, thread counts, and chunk sizes. Validates O2a and O2b across diverse workloads.	5 of 7 configs: disruption ≥ 0.88. 3+ configs: disruption > 1.0 (migration net-beneficial on proper NUMA topology).

A.4 Getting Started (Within 30 Minutes)

Verifies basic end-to-end functionality within 30 minutes.

Human time: ~5 minutes | Compute time: ~15 minutes

Step 1 — Verify instrumented kernel (2 min)

```
uname -r                                                  # must show 6.1.4
sudo ls /sys/kernel/debug/mig_timing                     # must exist
head -1 /sys/kernel/debug/mig_timing | tr ',' '\n' | wc -l  # must print 25
grep -c migrate_pages_deferred_ipi /proc/kallsyms   # must be ≥ 1
numactl --hardware                                        # 2 nodes, CPUs on both
```

Step 2 — Verify all files are present (30 sec)

```
ls ~/eval_scripts/
# Must show: run_x86_full.sh  run_t1.sh
#              mig_bench_x86.c  mig_bench_e_x86.c
#              analyze_unmap.py  analyze_timing.py  analyze_bench_e_x86.py
#              migrate.c  rmap.c  migrate_x86.patch  rmap_x86.patch  README
```

Step 3 — Run quick evaluation (~15 min)

```
cd ~/eval_scripts
echo 0 | sudo tee /proc/sys/kernel/numa_balancing
sudo bash run_x86_full.sh --quick --results-dir ./results_x86
# --quick: skips T1 and runs only Bench E config 0 (~55 sec) instead of all 7
```

Step 4 — Verify key claims (30 sec)

```
head -1 results_x86/timing_4kb_2048.csv | tr ',' '\n' | wc -l
# Must print: 25  (rmap patch active)

grep 'CONFIRMED' results_x86/run_x86.log
# Must show:
# ✓ O1 CONFIRMED: Batching significantly reduces TLB shootdown impact.
# ✓ O3 CONFIRMED: High locality in page-table walk.
```

Step 5 — Supply your own inputs

```
# Run any T-test on a custom timing CSV:
python3 analyze_unmap.py --t5 /path/to/any_timing.csv

# Compare quiescent vs loaded (T4):
python3 analyze_unmap.py --t4 results_x86/timing_4kb_2048.csv \
    results_x86/bench_e_timing_x86_E_rand_rmw_t4_c512.csv

# Run a specific Bench E config by index (0-based):
cd results_x86 && sudo ./mig_bench_e 5   # config 5 = rand_rmw_t4_c1
```

Appendix B : ARM64 ROCM Artifact

Artifact for ROCM (Read-Only Copy Migration): eliminates migration-entry stall for read-only file-backed pages. Evaluated on ARM64 Linux 6.1.4 with emulated NUMA (numa=fake=2).

B.1 File Inventory

All files must be placed in one flat directory alongside run_arm64_full.sh.

File	Size	Description
`run_arm64_full.sh`	~10 KB	Main automation script. 9 steps: preflight → build → Bench A-D → timing plots → stage comparison → Bench E → ROCM benchmark → summary. Run as: sudo bash run_arm64_full.sh [--quick] [--skip-rocm]
`mig_bench.c`	~35 KB	Userspace Benchmarks A, B, C, D (ARM64). Bench D uses CNTVCT_EL0 (ARM64 system counter, 24 MHz) for stall measurement instead of x86 RDTSC.
`mig_bench_e.c`	~35 KB	Benchmark E: 7 concurrent-load configurations (ARM64). Same 7 configs as x86 variant. Workers clamp to node 1 CPU count (≤2 on tested VM).
`mig_bench_rocm.c`	~15 KB	ROCM vs standard single-threaded comparison. Migrates 2048 file-backed read-only pages; records rocm_path flag and ro_swap_ns per page.
`mig_bench_rocm_mt.c`	~18 KB	ROCM vs standard multithreaded controlled test. 2 workers, identical random-read workload; only the kernel migration pipeline differs between runs.
`analyze_timing.py`	~31 KB	Stage breakdown analysis and PNG plots for Bench A/B/C CSVs. Same script as x86 variant.
`analyze_downtime.py`	~12 KB	Downtime analysis and PNG plots for Bench D CSVs. Handles ARM64 24 MHz timer resolution.

`migrate_rocm.c`	~96 KB	ROCM-patched mm/migrate.c. Contains ro_copy_and_swap(), folio_rmap_all_readonly() gate, and rocm_path/ro_swap_ns columns in mig_timing_record. Copy to linux-6.1.4/mm/migrate.c before building ROCM kernel.

B.2 Setup Instructions

Component	Requirement
CPU	ARM64 (AArch64), ≥4 cores. Tested: QEMU/KVM ARM64 VM with 4 vCPUs (2 per NUMA node). Extra cores reduce kernel build time only.
Memory	Minimum 1.5 GB. Recommended 2 GB to avoid swap during Bench E (512 MB buffer allocation). Kernel build: ~256 MB.
Storage	/ (root): 8 GB — OS, source, build output. /boot: 256 MB — two kernel images (baseline + ROCM). /tmp: 512 MB — build temps. Total: ~9 GB.
Extra HW	None. All experiments run in software on a single machine with emulated NUMA.
OS	Ubuntu 22.04 LTS ARM64 (Jammy Jellyfish). Other Debian-based ARM64 distributions should work.
Dependencies	sudo apt install -y build-essential libnuma-dev numactl python3-numpy python3-matplotlib
Boot param	Add to GRUB_CMDLINE_LINUX: numa=fake=2. Then: sudo update-grub && sudo reboot. Verify: numactl --hardware shows 2 nodes (0-1).

Kernel Compilation

```
# Baseline kernel (Benchmarks A-E)  Human: ~5 min | Compute: 8-35 min
cd ~/linux-6.1.4
cp /boot/config-$(uname -r) .config && make olddefconfig
make -j$(nproc)
sudo make modules_install && sudo make install && sudo reboot
uname -r                            # 6.1.4
ls /sys/kernel/debug/mig_timing     # must exist
```

```
# ROCM kernel (Benchmark ROCM only)  Human: ~5 min | Compute: ~2 min incremental
cp /path/to/artifact/migrate_rocm.c ~/linux-6.1.4/mm/migrate.c
cd ~/linux-6.1.4 && make -j$(nproc)   # only migrate.o recompiles
sudo make modules_install && sudo make install && sudo reboot
head -1 /sys/kernel/debug/mig_timing | grep rocm_path  # must match
```

Build Userspace Benchmarks

```
# Human time: < 1 minute
gcc -O2 -Wall -o mig_bench    mig_bench.c   -lnuma -lpthread
gcc -O2 -Wall -o mig_bench_e    mig_bench_e.c      -lnuma -lpthread -lm
# ROCM benchmarks (requires ROCM kernel):
gcc -O2 -Wall -o mig_bench_rocm    mig_bench_rocm.c       -lnuma -lm
gcc -O2 -Wall -o mig_bench_rocm_mt mig_bench_rocm_mt.c -lnuma -lm
```

B.3 Features and Evaluation Table

For each feature: test scenario, parameters, pointer to automation script, objective, and expected outcome.

Experiment	Threads	Parameters	Script	Objective	Expected outcome
Bench A 4KB quiescent	1 migrator	512 pages 2048 pages node 0→1 no concurrent load	run_arm64_full.sh Step 3 sudo ./mig_bench	Establish per-stage cost ratios on ARM64. Confirm copy-dominated profile. Produce quiescent baseline for all comparisons.	timing_4kb_2048.csv: 2048 records. Copy 38–46%, Unmap 38–42%. Mean total 1,500–2,200 ns/page.
Bench B 2MB THP	1 migrator	32 THPs 128 THPs node 0→1	Step 3 (auto) sudo ./mig_bench	Copy dominance at 2MB scale. Note: kernel splits THPs on numa=fake=2 VM — analytical projection substituted.	timing_2mb_*.csv. Copy ~95.8% real HW; analytical projection on VM.
Bench C Shared pages	1 migrator	64 pages deg 1,2,4,8,16,32,64 node 0→1	Step 3 (auto) sudo ./mig_bench	Measure rmap walk O(mapcount) scaling. Confirm unmap grows linearly with sharing degree.	7 CSVs timing_shared_deg001..064.csv. α ≈ 0.53 µs per additional sharing process.

Bench D Downtime	1 accessor 1 migrator	400,000 CNTVCT_EL0 samples same-node + cross-node 5,000 ns threshold	Step 3 (auto) sudo ./mig_bench	Measure app-visible stall from migration PTE. ARM64 timer: 24 MHz (41.67 ns/tick). Motivates ROCM.	downtime_samenode.csv: max stall ~62–80 µs. Normal access mean ~20–36 ns.
Bench E config 0 rand_rmw_t4_c512	2 workers (clamped) 1 migrator	512 MB buffer chunk=512 pages 10s/30s/10s	Step 6 run_arm64_full.sh --quick or: sudo ./mig_bench_e 0	Primary disruption metric for ARM64 standard kernel. Confirm near-zero disruption.	Disruption factor ≈ 1.094. p999 spike < 2×. Migration rate ~39,957 pages/s.
Bench E config 6 rand_rmw_t4_c1	2 workers 1 migrator	512 MB buffer chunk=1 page 10s/30s/10s	Step 6 sudo ./mig_bench_e 5	Show per-page overhead pathology. One syscall per page; no batch benefit. High p999 spike characteristic.	Disruption ≈ 0.962. p999 spike ~22.9×. Migration rate ~18,170 pages/s.
Bench E all 7	1–2 workers 1 migrator	7 configs sweep rand/seq × rmw/read t1/t4/t8 × c1/c64/c512	Step 6 full sudo ./mig_bench_e	Full sweep across patterns, thread counts, chunk sizes. Validates ARM64 standard-kernel low disruption profile.	5+ configs: disruption ≥ 1.0. c1 unique: p999 spike ~22×. All 131,072 pages migrate successfully.
ROCM Single-thread	1 migrator (file-backed pages)	2048 read-only file-backed pages node 0→1	Step 8 sudo ./mig_bench_rocm Requires ROCM kernel	Validate stall window 1,100→462 ns. Confirm rocm_path=1 in all fast-path records. Confirm remap_ns ≈ 0.	rocm_path=1: 8192/8192 records. remap_ns=0. ro_swap_ns ≈ 400–450 ns. rocm_comparison.txt generated.

ROCM Multithreaded	2 workers 1 migrator	2048 pages 15s/15s/15s phases anon (standard) vs file (ROCM)	Step 8 sudo ./mig_bench_rocm_mt Requires ROCM kernel	Controlled comparison: identical worker workload, only pipeline differs. Standard vs ROCM disruption and stall duration.	Standard DF 1.020 vs ROCM DF 1.077. Mean stall: 54,869 ns → 26,729 ns (2.05× shorter).

B.4 Getting Started (Within 30 Minutes)

Verifies basic end-to-end functionality within 30 minutes.

Human time: ~5 minutes | Compute time: ~15 minutes

Step 1 — Verify instrumented kernel (2 min)

```
uname -r                              # must show 6.1.4
sudo ls /sys/kernel/debug/mig_timing  # must exist
numactl --hardware                    # 2 nodes (0-1) with CPUs on both
# If testing ROCM kernel:
head -1 /sys/kernel/debug/mig_timing | grep rocm_path  # must match
```

Step 2 — Build and run a quick migration (3 min)

```
gcc -O2 -Wall -o mig_bench mig_bench.c -lnuma -lpthread
echo 0 | sudo tee /proc/sys/kernel/numa_balancing
sudo ./mig_bench
# Expected: Succeeded: 2048/2048   Failed: 0
#       Timing data → timing_4kb_2048.csv
```

Step 3 — Verify ARM64 stage profile (1 min)

```
python3 analyze_timing.py timing_4kb_2048.csv
# Expected: Copy 38-46%,  Unmap 38-42%
# If Copy < 20%: wrong kernel booted — check uname -r
```

Step 4 — Run quick automated evaluation (~10 min)

```
sudo bash run_arm64_full.sh --quick
# Runs: A+B+C+D → analyze → Bench E config 0 → ROCM (if ROCM kernel)

# Verify ROCM result (ROCM kernel only):
grep ',1$' results_arm64/rocm_fast_timing.csv | wc -l  # must be > 0
cat results_arm64/rocm_comparison.txt
```

Step 5 — Supply your own inputs

```
# Run a specific Bench E config by index (0-based):
sudo ./mig_bench_e 0   # E_rand_rmw_t4_c512 (primary config)
sudo ./mig_bench_e 5   # E_rand_rmw_t4_c1   (chunk=1, max overhead)

# Reset ring buffer and capture fresh sample:
echo 1 | sudo tee /sys/kernel/debug/mig_timing
sudo ./mig_bench
python3 analyze_timing.py timing_4kb_2048.csv
```